SAE
L. Bung

# Webentwicklung: Webserver

Klasse: E3FI
Datum:

## Aufgabe 1: Der erste eigene Webserver

~ 15 min.

Bisher habt ihr Anfragen an bestehende Webserver und REST-APIs gestellt. Heute wechselt ihr die Seite und programmiert selbst einen Webserver. *Flask* ist ein schlankes Python-Framework, mit dem sich Webserver und REST-APIs mit wenigen Zeilen Code erstellen lassen. Die offizielle Dokumentation findet ihr unter `https://flask.palletsprojects.com`.

a) Legt einen neuen Projektordner an und installiert Flask im Terminal mit folgendem Befehl:

```
python -m pip install flask
```

Im Schulnetz sollte Flask bereits installiert sein.

b) Erstellt in diesem Ordner die Datei `app.py` mit folgendem Inhalt:

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def index():
7     return "Hallo Welt!"
```

c) Startet den Server im Terminal mit dem folgenden Befehl und ruft anschließend `http://127.0.0.1:5000` im Browser auf:

```
flask --app app run --debug
```

d) Erklärt die Bedeutung der Zeile `@app.route(/")`.

e) Ändert den zurückgegebenen Text, speichert die Datei und ladet die Seite im Browser neu. Beschreibt, welche Aufgabe die Option `--debug` hat.

## Aufgabe 2: Routen und Parameter

~ 15 min.

Ein Webserver kann auf unterschiedliche URLs mit unterschiedlichen Inhalten reagieren. Teile der URL können dabei als Parameter an eine Funktion übergeben werden.

a) Ergänzt eine Route `/info`, die einen kurzen Text über euren Ausbildungsbetrieb zurückgibt.

b) Ergänzt die folgende Route und testet sie mit verschiedenen Namen, z. B. `/hallo/Alex`:

```
@app.route("/hallo/<name>")
def hallo(name):
    return f"Hallo {name}!"
```

c) Erstellt eine Route `/quadrat/<int:zahl>`, die das Quadrat der übergebenen Zahl zurückgibt. Erklärt, was passiert, wenn statt einer Zahl ein Wort übergeben wird.

d) Ruft eine URL auf, für die es keine Route gibt. Notiert den Statuscode, den der Server zurückliefert.

## Aufgabe 3: Eine Webseite ausliefern

~ 15 min.

Bisher liefert euer Server nur einfachen Text. Ein Webserver kann aber auch vollständige Webseiten aus HTML, CSS und JavaScript ausliefern. Flask erwartet HTML-Dateien dabei im Ordner `templates` und CSS- sowie JavaScript-Dateien im Ordner `static`.

a) Legt in eurem Projektordner die folgende Ordnerstruktur an:

```
projekt/
    app.py
    templates/
        index.html
    static/
        style.css
        script.js
```

b) Ersetzt in `app.py` die bisherige Route `/` durch folgenden Code:

```
from flask import Flask, render_template
from datetime import datetime
app = Flask(__name__)
@app.route("/")
def index():
    return render_template("index.html")
@app.route("/zeit")
def zeit():
    return datetime.now().strftime("%H:%M:%S")
```

Übernehmt eure Routen aus Aufgabe 2 unverändert.

c) Füllt die Datei `index.html` mit folgendem Inhalt:

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <title>Mein Flask-Server</title>
  <link rel="stylesheet"
        href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <h1>Willkommen auf meinem Server!</h1>
  <button id="knopf">Serverzeit abfragen</button>
  <p id="ausgabe"></p>
  <script src="{{ url_for('static', filename='script.js') }}"></
     script>
</body>
</html>
```

d) Gestaltet die Seite in der Datei `style.css` mit eigenen CSS-Regeln.

e) Schreibt in der Datei `script.js` ein Skript, das beim Klick auf den Button mit `fetch` die Route `/zeit` abfragt und die Antwort im Absatz `ausgabe` anzeigt.

f) Ladet die Seite neu und beobachtet den Reiter „Netzwerk“ in den Entwicklertools (Taste F12). Erklärt, warum beim Aufruf einer einzigen Seite mehrere Anfragen an den Server gestellt werden.

g) Erklärt, welchen Vorteil `url_for` gegenüber einem fest eingetragenen Pfad wie `/static/style.css` haben kann.

## Aufgabe 4: Eine eigene REST-API

~ 30 min.

Eine REST-API liefert keine fertigen Webseiten, sondern Daten, meist im JSON-Format. Ihr entwickelt nun eine API zur Verwaltung einer Aufgabenliste. Die Daten werden dabei zunächst nur in einer Python-Liste im Arbeitsspeicher gehalten.

a) Erstellt im selben Projektordner eine neue Datei `api.py` mit folgendem Grundgerüst:

```python
from flask import Flask, jsonify, request
app = Flask(__name__)
aufgaben = [
    {"id": 1, "titel": "Flask installieren", "erledigt": True},
    {"id": 2, "titel": "REST-API bauen", "erledigt": False},
]
@app.get("/api/aufgaben")
def alle_aufgaben():
    return jsonify(aufgaben)
@app.get("/api/aufgaben/<int:aufgabe_id>")
def eine_aufgabe(aufgabe_id):
    # TODO: passende Aufgabe suchen und zurueckgeben
    # TODO: falls nicht vorhanden: Fehlermeldung mit Statuscode 404
    pass
@app.post("/api/aufgaben")
def aufgabe_anlegen():
    daten = request.get_json()
    neue_aufgabe = {
        "id": max((a["id"] for a in aufgaben), default=0) + 1,
        "titel": daten["titel"],
        "erledigt": False,
    }
    aufgaben.append(neue_aufgabe)
    return jsonify(neue_aufgabe), 201
```

b) Beendet den laufenden Server mit Strg+C. Startet anschließend die API mit `flask --app api run --debug` und ruft `/api/aufgaben` im Browser auf.

c) Vervollständigt die Funktion `eine_aufgabe`. Wird keine Aufgabe mit der gesuchten ID gefunden, soll der Server mit `jsonify({"fehler: "Nicht gefunden"}), 404` antworten.

d) Erklärt, warum die Funktion `aufgabe_anlegen` den Statuscode 201 statt 200 zurückgibt.

e) Erklärt, warum POST-Anfragen nicht einfach über die Adresszeile des Browsers getestet werden können.

## Aufgabe 5: Die API testen

~ 15 min.

Um auch POST-Anfragen an eure API zu senden, benötigt ihr einen HTTP-Client. Ihr könnt dafür z. B. eine Erweiterung wie *Thunder Client* oder *REST Client* in VS Code, das Programm *Postman* oder ein Python-Skript verwenden.

a) Erstellt eine weitere Datei `client.py`, die bei laufendem Server eine neue Aufgabe anlegt:

```
1 import requests
2 url = "http://127.0.0.1:5000/api/aufgaben"
3 antwort = requests.post(url, json={"titel": "API testen"})
4 print(antwort.status_code)
5 print(antwort.json())
```

Installiert die Bibliothek dazu gegebenenfalls mit `python -m pip install requests`.

b) Führt das Skript mehrmals aus und überprüft anschließend im Browser, ob die neuen Aufgaben in der Liste erscheinen.

c) Erweitert das Skript so, dass es zusätzlich eine einzelne Aufgabe sowie eine nicht vorhandene Aufgabe abfragt und jeweils den Statuscode ausgibt.

d) Startet den Server neu und ruft die Liste erneut ab. Erklärt eure Beobachtung und nennt eine Möglichkeit, das Problem zu lösen.

## ⊕ Bonusaufgabe 6: Vertiefungsaufgaben

Die folgenden Aufgaben sind unabhängig voneinander und können in beliebiger Reihenfolge bearbeitet werden.

a) **Eingaben prüfen:** Sendet eine POST-Anfrage ohne das Feld `titel` und beschreibt, was passiert. Erweitert die Funktion `aufgabe_anlegen` so, dass der Server in diesem Fall mit einer verständlichen Fehlermeldung und dem Statuscode 400 antwortet.

b) **Filtern mit Query-Parametern:** Erweitert die Route `/api/aufgaben` so, dass mit `/api/aufgaben?erledigt=true` nur erledigte Aufgaben zurückgegeben werden. Recherchiert dazu die Verwendung von `request.args` in Flask.

c) **Frontend für die API:** Ergänzt in `api.py` eine Route `/`, die wie in Aufgabe 3 eine neue Seite `aufgaben.html` aus dem Ordner `templates` ausliefert. Diese Seite soll die Aufgaben per `fetch` von eurer API laden und als Liste anzeigen. Ergänzt ein HTML-Formular, mit dem neue Aufgaben angelegt werden können. Wer möchte, gestaltet die Seite mit Bootstrap.

d) **Dauerhafte Speicherung:** Sorgt dafür, dass die Aufgaben einen Neustart des Servers überstehen. Speichert sie dazu entweder in einer JSON-Datei oder in einer SQLite-Datenbank mit dem Python-Modul `sqlite3`.

e) **Vollständiges CRUD:** Ergänzt eine Route mit der Methode PUT, mit der eine Aufgabe geändert werden kann (z. B. als erledigt markieren). Ergänzt außerdem eine Route mit der Methode DELETE, mit der eine Aufgabe gelöscht werden kann. Achtet jeweils auf passende Statuscodes.